

MINISTÈRE DE L'ENSEIGNEMENT
SUPÉRIEUR ET DE LA RECHERCHE
(MESR)

REPUBLIQUE TOGOLAISE
Travail-Liberté-Patrie



ECOLE SUPÉRIEURE DE GESTION D'INFORMATIQUE ET DES SCIENCES

RAPPORT DE PROJET

DOMAINE: IA-BIG DATA

MENTION: Introduction à AWS

SPÉCIALITÉ: Intelligence Artificielle

THÈME: Pipeline d'Ingestion de Données IoT en Temps Réel (Serverless)

Rédigée par :
SENOU Audrey

Professeur:
Mofiala Hervé LOKOSSOU

ANNÉE ACADÉMIQUE: 2025-2026

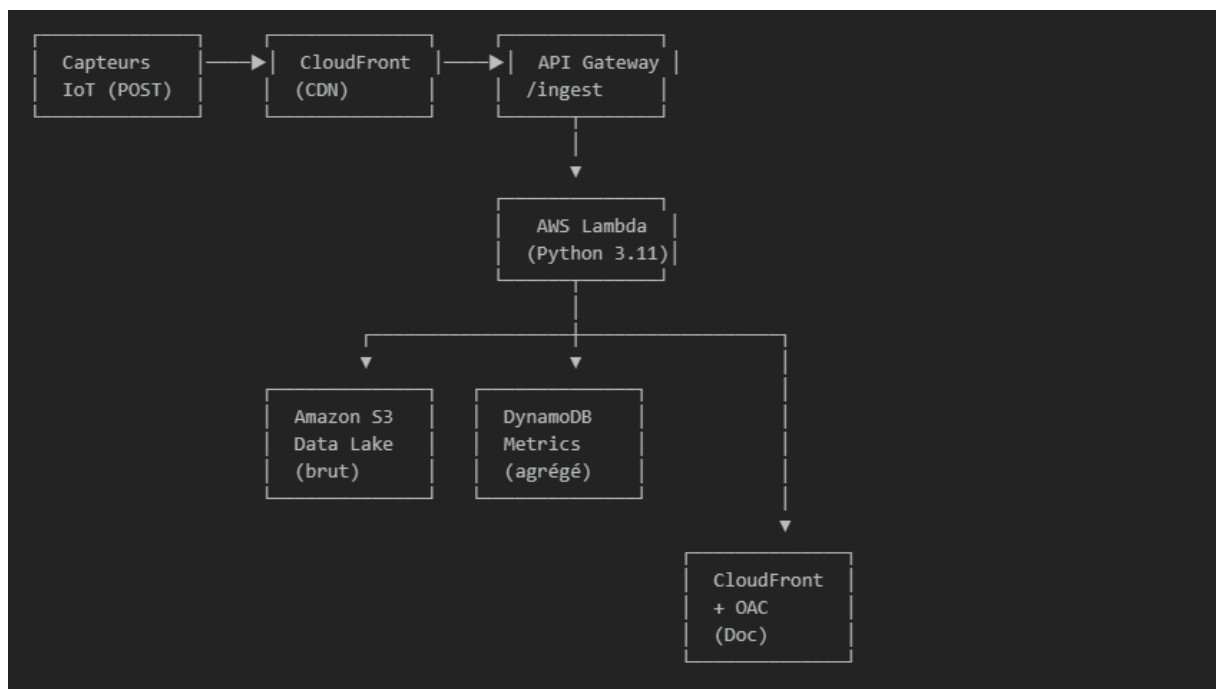
SOMMAIRE

Introduction.....	3
Présentation de l'architecture.....	3
Services AWS utilisés.....	4
Infrastructure as Code (CloudFormation).....	5
Déploiement de l'Infrastructure.....	7
Captures d'Écran.....	13
Conclusion.....	16
Table des matières.....	20

Introduction

Dans le cadre du cours d'Introduction à AWS, ce projet a pour objectif de mettre en place une architecture cloud serverless permettant l'ingestion en temps réel de données provenant de capteurs IoT. L'architecture s'appuie sur plusieurs services AWS tels que CloudFormation, CloudFront, API Gateway, Lambda, S3, DynamoDB et CloudWatch. L'ensemble de l'infrastructure est déployé automatiquement à l'aide d'Infrastructure as Code (IaC).

Présentation de l'architecture



Description de l'architecture mise en place

L'architecture développée repose sur une approche **serverless** et **hautement scalable** permettant l'ingestion et le traitement en temps réel des données issues de capteurs IoT.

Les capteurs IoT envoient leurs mesures sous forme de requêtes HTTP POST. Ces requêtes transitent d'abord par une distribution **Amazon CloudFront**, qui agit comme

un réseau de diffusion de contenu (CDN) afin d'améliorer les performances et de réduire la latence pour les équipements situés dans différentes régions géographiques.

Les requêtes sont ensuite transmises à **Amazon API Gateway**, qui expose le point d'entrée **/ingest** et assure la réception sécurisée des données. API Gateway déclenche alors l'exécution d'une fonction **AWS Lambda** développée en Python 3.11.

La fonction Lambda constitue le cœur du traitement. Elle analyse le contenu JSON reçu, enregistre les données brutes dans un **Data Lake Amazon S3** en utilisant une stratégie de partitionnement temporel, puis calcule plusieurs indicateurs tels que la température moyenne et le nombre d'anomalies détectées.

Les données brutes sont conservées dans **Amazon S3**, qui joue le rôle de Data Lake. Cette couche de stockage permet de conserver l'ensemble des informations collectées pour de futurs traitements analytiques ou projets de Data Science.

Les métriques agrégées produites par la fonction Lambda sont enregistrées dans **Amazon DynamoDB**, qui sert de couche de consultation rapide (Serving Layer). Cette base NoSQL permet d'accéder rapidement aux indicateurs calculés sans avoir à parcourir l'ensemble des données brutes stockées dans S3.

Parallèlement, une seconde infrastructure est dédiée à la documentation technique du projet. Les fichiers HTML sont stockés dans un bucket S3 privé et sont publiés via une seconde distribution **Amazon CloudFront**. L'accès direct au bucket est interdit grâce à l'utilisation d'un **Origin Access Control (OAC)**, garantissant que les utilisateurs ne peuvent accéder à la documentation qu'à travers CloudFront.

Enfin, **Amazon CloudWatch** assure la supervision de l'ensemble du système en collectant les journaux d'exécution de la fonction Lambda, facilitant ainsi le suivi des traitements et le diagnostic des erreurs éventuelles.

Services AWS utilisés

Service	Rôle
AWS CloudFormation (SAM)	Infrastructure as Code
Amazon S3 (x2)	Data Lake brut + Documentation
Amazon DynamoDB	Feature Store / Métriques agrégées

AWS Lambda	Traitement à la volée (Python 3.11)
Amazon API Gateway	Point d'entrée HTTP REST
Amazon CloudFront (x2)	CDN ingestion + CDN documentation sécurisé
Amazon CloudWatch	Logs et monitoring
AWS IAM	Gestion des permissions

Infrastructure as Code (CloudFormation)

Fichier : [infrastructure/template.yaml](#)

Le template SAM (Serverless Application Model) définit toutes les ressources :

3.1 Buckets S3

- **RawDataBucket** (`${AWS::StackName}-raw-data`) : Stockage des données brutes IoT
- **DocumentationBucket** (`${AWS::StackName}-tech-doc`) : Hébergement du site de documentation

3.2 Table DynamoDB

- **MetricsTable** (`${AWS::StackName}-metrics`) : Stockage des métriques agrégées
 - Clé primaire : `request_id` (String)
 - Mode facturation : `PAY_PER_REQUEST`

3.3 Fonction Lambda

- **IngestionFunction** (`iot-ingestion-${Environment}`)
 - Runtime : Python 3.11
 - Handler : `src/index.handler`
 - Mémoire : 256 MB
 - Timeout : 30 secondes
 - Variables d'environnement : `S3_BUCKET`, `DYNAMODB_TABLE`
 - Politiques : `S3CrudPolicy` + `DynamoDBCrudPolicy`

3.4 API Gateway HTTP

- **HttpApi** : API HTTP (ProtocolType: HTTP)
- **IngestionRoute** : POST `/ingest` avec intégration Lambda proxy
- **ApiStage** : Stage `$default` avec `AutoDeploy`

3.5 CloudFront Ingestion

- **IngestionDistribution** : Distribution CloudFront devant l'API Gateway
 - Origine : API Gateway avec CustomOriginConfig (HTTPS)
 - CachePolicyId : 4135ea2d-6df8-44a3-9df3-4b5a84be39ad (CachingDisabled)

3.6 CloudFront Documentation + OAC

- **DocumentationOAC** : Origin Access Control pour le bucket de documentation
 - SigningBehavior : always
 - SigningProtocol : sigv4
- **DocumentationBucketPolicy** : Bucket Policy autorisant uniquement CloudFront
- **DocumentationDistribution** : Distribution CloudFront pour le site de documentation
 - DefaultRootObject : index.html
 - OriginAccessControlId : référence vers l'OAC

3.7 Outputs

Output	Description
CloudFrontIngestionURL	URL CDN pour l'ingestion IoT
CloudFrontDocURL	URL CDN pour la documentation
DataLakeBucket	Nom du bucket Data Lake
DocumentationBucketName	Nom du bucket documentation
DynamoDBTable	Nom de la table DynamoDB
ApiEndpoint	URL directe de l'API Gateway

Déploiement de l'Infrastructure

Prérequis

Avant le déploiement de l'infrastructure, plusieurs outils et prérequis étaient nécessaires :

- AWS CLI installée et configurée avec un compte AWS disposant des autorisations requises.
- AWS SAM CLI installée afin de faciliter le déploiement des ressources serverless.
- Python 3.11 installé pour le développement et les tests de la fonction Lambda.
- Permissions IAM permettant la création et la gestion des services AWS utilisés dans le projet, notamment CloudFormation, Lambda, API Gateway, DynamoDB, S3 et CloudFront.

Ces prérequis ont permis d'assurer un déploiement automatisé et reproductible de l'ensemble de l'architecture.

Construction de l'Application Serverless

La première étape a consisté à construire l'application à partir du template CloudFormation développé avec AWS SAM. Cette opération permet de préparer les ressources et les dépendances nécessaires au déploiement.

La commande suivante a été exécutée :

```
sam build -t infrastructure/template.yaml
```

Cette phase génère automatiquement les artefacts nécessaires dans le répertoire

```
.aws-sam/build.
```

Déploiement Initial

Une fois la phase de construction terminée, l'infrastructure a été déployée sur AWS à l'aide de la commande :

```
sam deploy --guided
```

Lors du déploiement, les paramètres suivants ont été renseignés :

Paramètre	Valeur
Stack Name	devoir-iot-ksenou
Région AWS	eu-west-3 (Paris)
Environment	ksenou
Confirm Changeset	Yes
CAPABILITY_IAM	Yes

AWS SAM a alors créé automatiquement l'ensemble des ressources définies dans le fichier `template.yaml`, notamment les buckets S3, la table DynamoDB, la fonction Lambda, l'API Gateway ainsi que les distributions CloudFront.

Mise à Jour de l'Infrastructure

Après certaines modifications apportées au code ou à l'infrastructure, la pile CloudFormation a été mise à jour sans nécessiter une recréation complète des ressources.

La commande suivante a été utilisée :

```
sam deploy --resolve-s3 --region eu-west-3 --stack-name
devoir-iot-ksenou --template-file .aws-sam/build/template.yaml
--no-fail-on-empty-changeset --parameter-overrides Environment=ksenou
--capabilities CAPABILITY_IAM
```

Résultat attendu

CloudFormation outputs from deployed stack

```
Key                DataLakeBucket
Description        -
Value              devoir-iot-ksenou-raw-data
```

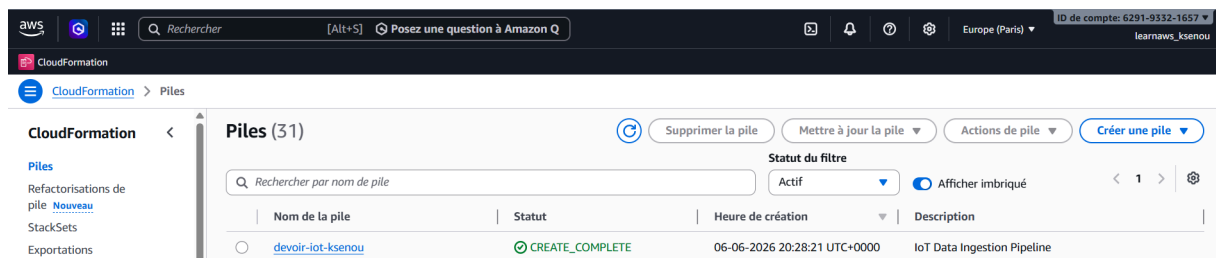

Key	DocumentationBucketName
Description	-
Value	devoir-iot-ksenou-tech-doc
Key	ApiEndpoint
Description	URL directe de l'API
Value	https://d1ngb7eee3.execute-api.eu-west-3.amazonaws.com
Key	CloudFrontIngestionURL
Description	URL d'ingestion
Value	https://dimjaatmzr2n.cloudfront.net
Key	CloudFrontDocURL
Description	URL documentation
Value	https://dfzv9amaast7t.cloudfront.net
Key	DynamoDBTable
Description	-
Value	devoir-iot-ksenou-metrics

Cette approche permet d'appliquer uniquement les changements nécessaires tout en conservant les ressources déjà déployées.

Validation du Déploiement

À l'issue du déploiement, la pile CloudFormation est passée avec succès à l'état `CREATE_COMPLETE`, confirmant la création correcte de toutes les ressources de l'architecture.

Capture d'écran n°1 : Pile CloudFormation au statut `CREATE_COMPLETE`



Vérification des Outputs

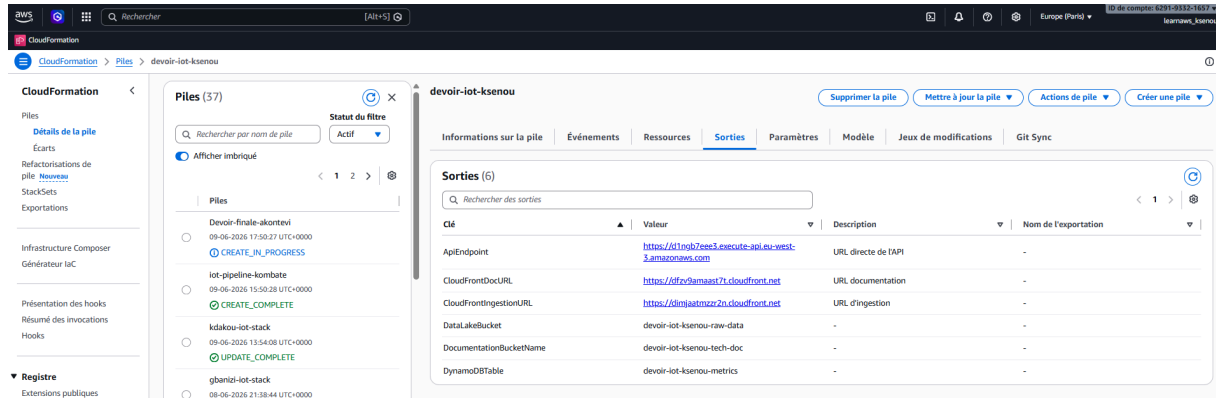
Une fois le déploiement terminé, CloudFormation a généré plusieurs informations de sortie (Outputs) permettant d'accéder aux différents composants du système.

Les principales valeurs récupérées sont :

Output	Description
CloudFrontIngestionURL	URL publique utilisée pour l'ingestion des données IoT
CloudFrontDocURL	URL publique du site de documentation technique
DataLakeBucket	Bucket S3 contenant les données brutes
DocumentationBucketName	Bucket S3 hébergeant la documentation
DynamoDBTable	Table DynamoDB stockant les métriques agrégées
ApiEndpoint	URL directe de l'API Gateway

Ces sorties ont été utilisées pour réaliser les tests d'ingestion, l'accès à la documentation ainsi que les différentes vérifications fonctionnelles du projet.

Capture d'écran n°2 : Outputs générés par CloudFormation



Code Lambda (Traitement à la volée)

Fichier : `src/index.py`

Fonctionnalités

- **Parsing JSON** du corps de la requête HTTP
- **Partitionnement temporel S3** : `raw-zone/year=YYYY/month=MM/`
- **Calcul température moyenne** des mesures valides
- **Comptage des anomalies** (status = "ERROR")
- **Écriture DynamoDB** avec Decimal pour compatibilité
- **Gestion d'erreurs** avec Stack Trace dans CloudWatch

Script Client de Test

Fichier : `test_client.py`

Exécution: `python test_client.py`

Résultats attendus

- Test 1 : Status **201** — Ingestion réussie via CloudFront
- Test 2 : Status **201** — Ingestion réussie via API directe
- Test 3 : Status **400** — JSON corrompu détecté
- Test 4 : Status **201** — Températures manquantes (calcul avec 0 mesures)

Site de Documentation Technique

Fichier : `index.html`

Page HTML statique contenant :

- Titre du cours : "Ingénierie Big Data & IA — Architecture Cloud AWS"
- Description complète de l'architecture déployée
- Schéma du flux de données
- Composants avec leurs rôles
- Notes de sécurité

Upload vers S3

```
aws s3 cp index.html s3://devoir-iot-ksenou-tech-doc/index.html
```

Vérification de la sécurité OAC

```
# Accès direct S3 → doit retourner 403 Access Denied
```

```
curl -I
```

```
https://devoir-iot-ksenou-tech-doc.s3.eu-west-3.amazonaws.com/index.htm  
l
```

```
# Accès via CloudFront → doit retourner 200 OK
```

```
curl -I https://xxxxxxxxxxxxx.cloudfront.net/index.html
```

Validation et Tests

Vérification S3 (Data Lake)

```
aws s3 ls s3://devoir-iot-ksenou-raw-data/raw-zone/year=2026/month=06/
```

Résultat : Fichier JSON présent avec le partitionnement temporel

Vérification DynamoDB (Métriques)

```
aws dynamodb scan --table-name devoir-iot-ksenou-metrics --region  
eu-west-3
```

Résultat : Ligne insérée avec `request_id`, `timestamp`, `s3_path`, `avg_temperature`, `error_count`

Monitoring et Gestion des Échecs

9.1 CloudWatch Logs

```
# Lister les groupes de logs
```

```
aws logs describe-log-groups --region eu-west-3 --log-group-name-prefix  
/aws/lambda/iot-ingestion
```

```
# Lire les logs d'exécution
```

```
aws logs get-log-events \
```

```
--region eu-west-3 \
```

```
--log-group-name /aws/lambda/iot-ingestion-ksenou \
```

```
--log-stream-name "2026/06/09/[$LATEST]xxxxx"
```

9.2 Test d'échec (payload corrompu)

```
curl -X POST https://xxxxxxxxxxxxx.cloudfront.net/ingest \
```

```
-d "not valid json" \
```

```
-H "Content-Type: application/json"
```

Résultat : Status **400** avec message d'erreur + Stack Trace complète dans CloudWatch

Captures d'Écran

CloudFormation — Stack CREATE_COMPLETE

The screenshot shows the AWS CloudFormation console interface. At the top, there's a navigation bar with the AWS logo, a search bar, and account information. The main content area is titled 'CloudFormation' and shows a list of stacks under the heading 'Piles (31)'. The stack 'devoir-iot-ksenou' is highlighted, showing its status as 'CREATE_COMPLETE' and its creation time as '06-06-2026 20:28:21 UTC+0000'. The description of the stack is 'IoT Data Ingestion Pipeline'.

Nom de la pile	Statut	Heure de création	Description
devoir-iot-ksenou	CREATE_COMPLETE	06-06-2026 20:28:21 UTC+0000	IoT Data Ingestion Pipeline

Data Lake avec fichiers

The screenshot shows the AWS IAM console interface. On the left, there's a navigation menu with options like 'Amazon S3', 'Compagniments', 'Fichiers', and 'Gestion des accès et sécurité'. The main area displays the 'month=06/' folder contents. A table lists four objects (JSON files) with columns for Name, Type, Dernière modification, Taille, and Classe de stockage. The objects are: 48797a77-3471-4985-a72c-5b35560e74.json, b35d7f0e-aac7-41e8-8b5d-c8b610c9715a.json, b35d7f0e-aac7-41e8-8b5d-c8b610c9715a.json, and c79f633b-6e68-4447-ae3b-a5259ea78222.json.

Nom	Type	Dernière modification	Taille	Classe de stockage
48797a77-3471-4985-a72c-5b35560e74.json	json	09 Jun 2026 02:32:53 PM GMT	502.0 o	Standard
b35d7f0e-aac7-41e8-8b5d-c8b610c9715a.json	json	09 Jun 2026 01:57:44 PM GMT	502.0 o	Standard
b35d7f0e-aac7-41e8-8b5d-c8b610c9715a.json	json	09 Jun 2026 04:21:46 PM GMT	502.0 o	Standard
c79f633b-6e68-4447-ae3b-a5259ea78222.json	json	09 Jun 2026 04:21:47 PM GMT	95.0 o	Standard

DynamoDB — Lignes insérées

The screenshot shows the AWS IAM console interface for DynamoDB. On the left, there's a navigation menu with options like 'DynamoDB', 'Tableau de bord', 'Tables', 'Explorer les éléments', 'Éditeur PartiQL', 'Sauvegarder', 'Exportations vers S3', 'Importations depuis S3', 'Intégrations', 'Capacité réservée', 'Paramètres', 'DAX', 'Clusters', 'Groupes de sous-réseau', 'Groupes de paramètres', and 'Événements'. The main area displays a query result for a query named 'Query 1'. The query is: SELECT * FROM "devicor-iot-kseu-metrics". The results are shown in a table with columns: error_count, total_measurements, avg_temperature, request_id, s3_path, and timestamp. The results are: 2, 5, 23.28, 48797a77-3..., raw-zone/fy..., 2026-06-09T14:52:52.000Z; 2, 5, 23.28, b35d7f0e-a..., raw-zone/fy..., 2026-06-09T16:21:45.000Z; 0, 1, 0, c79f633b-6e..., raw-zone/fy..., 2026-06-09T16:21:46.000Z.

error_count	total_measurements	avg_temperature	request_id	s3_path	timestamp
2	5	23.28	48797a77-3...	raw-zone/fy...	2026-06-09T14:52:52.000Z
2	5	23.28	b35d7f0e-a...	raw-zone/fy...	2026-06-09T16:21:45.000Z
0	1	0	c79f633b-6e...	raw-zone/fy...	2026-06-09T16:21:46.000Z

[illegible]

```

▶ 2026-06-09T16:21:44.446Z INIT_START Runtime version: python:3.11.main|nev2.v11 Runtime Version ARN: arn:aws:lambda:eu-west-3::runtime:53285ee27b9fac4cc9eb0b3c5ac1a45832297f81efabe2356a10a7e37355c64e
▶ 2026-06-09T16:21:45.006Z START RequestId: 3e18deea-c64f-4845-9746-1b85b67d9c8e Version: $LATEST
▶ 2026-06-09T16:21:45.237Z END RequestId: 3e18deea-c64f-4845-9746-1b85b67d9c8e
▶ 2026-06-09T16:21:45.237Z REPORT RequestId: 3e18deea-c64f-4845-9746-1b85b67d9c8e Duration: 230.57 ms Billed Duration: 787 ms Memory Size: 256 MB Max Memory Used: 98 MB Init Duration: 555.04 ms

```

```

▶ 2026-06-09T16:21:45.712Z      START RequestId: a3c33401-6723-4f7c-a721-6a81dfaa680b Version: $LATEST
▶ 2026-06-09T16:21:45.713Z      ERROR: Expecting value: line 1 column 1 (char 0)
▶ 2026-06-09T16:21:45.714Z      Traceback (most recent call last):
▶ 2026-06-09T16:21:45.714Z      File "/var/task/src/index.py", line 27, in handler
▶ 2026-06-09T16:21:45.714Z      body = json.loads(event["body"])
▶ 2026-06-09T16:21:45.714Z      ~~~~~
▶ 2026-06-09T16:21:45.714Z      File "/var/lang/lib/python3.11/json/__init__.py", line 346, in loads
▶ 2026-06-09T16:21:45.714Z      return _default_decoder.decode(s)
▶ 2026-06-09T16:21:45.714Z      ~~~~~
▶ 2026-06-09T16:21:45.714Z      File "/var/lang/lib/python3.11/json/decoder.py", line 337, in decode
▶ 2026-06-09T16:21:45.714Z      obj, end = self.raw_decode(s, idx=_w(s, 0).end())
▶ 2026-06-09T16:21:45.714Z      ~~~~~
▶ 2026-06-09T16:21:45.714Z      File "/var/lang/lib/python3.11/json/decoder.py", line 355, in raw_decode
▶ 2026-06-09T16:21:45.714Z      raise JSONDecodeError("Expecting value", s, err.value) from None
▶ 2026-06-09T16:21:45.714Z      json.decoder.JSONDecodeError: Expecting value: line 1 column 1 (char 0)
▶ 2026-06-09T16:21:45.716Z      END RequestId: a3c33401-6723-4f7c-a721-6a81dfaa680b

```

```
Successfully created/updated stack - devoir-iot-ksenou in eu-west-3

PS C:\Users\ksenou\Desktop\LABO_DEV\IADB\aws\DEVOIR> python test_client.py
=== Test 1: Valid payload via API Gateway direct ===
Status Code: 201
Response Body: {
  "message": "Ingestion successful",
  "request_id": "b35d7fce-aac7-41e8-985d-c89610ca716a",
  "s3_path": "raw-zone/year=2026/month=06/b35d7fce-aac7-41e8-985d-c89610ca716a.json",
  "avg_temperature": 23.28,
  "error_count": 2
}
✅ Ingestion successful!

=== Test 2: Corrupted payload (malformed JSON) ===
Status Code: 400
Response Body: {"message": "Ingestion failed", "error": "Expecting value: line 1 column 1 (char 0)"}
✅ Error correctly caught with stack trace in CloudWatch!

=== Test 3: Missing temperature values ===
Status Code: 201
Response Body: {
  "message": "Ingestion successful",
  "request_id": "c79fe33b-6ea8-4447-ae3b-a5259eaa7822",
  "s3_path": "raw-zone/year=2026/month=06/c79fe33b-6ea8-4447-ae3b-a5259eaa7822.json",
  "avg_temperature": 0.0,
  "error_count": 0
}
PS C:\Users\ksenou\Desktop\LABO_DEV\IADB\aws\DEVOIR>
```

Conclusion

Ce projet a permis de mettre en œuvre une architecture serverless complète sur AWS pour l'ingestion de données IoT en temps réel. L'utilisation de CloudFormation a permis d'automatiser le déploiement de l'infrastructure tandis que Lambda, S3 et DynamoDB ont assuré le traitement et le stockage des données.

Les tests réalisés ont confirmé le bon fonctionnement de la chaîne d'ingestion ainsi que la capacité du système à gérer les erreurs et à produire des métriques exploitables. La sécurisation de la documentation grâce à CloudFront et OAC a également permis de respecter les bonnes pratiques AWS en matière de sécurité.

Cette architecture constitue une base solide pour le développement futur de solutions Big Data capables de traiter des volumes de données beaucoup plus importants.

A. Réponses aux Questions Théoriques

Question 1 : Qu'est-ce que l'Infrastructure as Code (IaC) et quel est le rôle d'AWS CloudFormation ?

L'**Infrastructure as Code (IaC)** consiste à créer et gérer l'infrastructure informatique (serveurs, réseaux, bases de données, etc.) à l'aide de fichiers de configuration plutôt que manuellement.

AWS CloudFormation est un service qui permet de définir cette infrastructure sous forme de modèles (templates) et de la déployer automatiquement. Cela facilite la création, la modification et la suppression des ressources AWS tout au long du cycle de vie d'une application.

Question 2 : Qu'est-ce qu'une fonction AWS Lambda ? Quelle différence avec Amazon EC2 ?

Une **fonction AWS Lambda** est un programme qui s'exécute automatiquement lorsqu'un événement se produit (arrivée d'un fichier, appel API, etc.).

L'approche **Serverless** de Lambda signifie que AWS gère les serveurs à la place du développeur.

Différences avec EC2 :

- **Lambda** : pas de serveur à gérer, paiement uniquement lorsque le code s'exécute.
- **EC2** : il faut créer, configurer et maintenir des serveurs virtuels qui fonctionnent en permanence.

Question 3 : Pourquoi placer CloudFront devant API Gateway pour une collecte mondiale de données IoT ?

Amazon CloudFront est un réseau de distribution de contenu qui possède des points de présence partout dans le monde.

En le plaçant devant **API Gateway** :

- les données sont reçues plus rapidement ;
- la latence est réduite ;
- les performances sont améliorées pour les objets IoT situés dans différents pays ;
- l'architecture devient plus scalable.

Question 4 : Pourquoi stocker les données brutes dans S3 et les indicateurs dans DynamoDB ?

Amazon S3 est idéal pour stocker de très grandes quantités de données brutes à faible coût.

Amazon DynamoDB est optimisé pour fournir rapidement des données déjà traitées et agrégées.

Cette séparation permet :

- de réduire les coûts ;
- d'améliorer les performances ;
- de mieux gérer les gros volumes de données.

Une base de données relationnelle unique serait plus coûteuse et moins performante pour ce type d'usage Big Data.

Question 5 : Expliquez le modèle de responsabilité partagée d'AWS pour Amazon S3.

Le modèle de responsabilité partagée signifie que :

- **AWS** protège l'infrastructure physique (serveurs, réseaux, centres de données).
- **Le client** protège ses données en configurant correctement les accès, les permissions et le chiffrement.

Ainsi, AWS sécurise le cloud, tandis que le client sécurise ce qu'il met dans le cloud.

Question 6 : Pourquoi éviter le "Static Website Hosting" public pour une documentation interne ?

Activer le **Static Website Hosting** public rend les fichiers accessibles à tout le monde sur Internet.

Pour une documentation interne, cela représente un risque de fuite d'informations.

Avec **CloudFront + Origin Access Control (OAC)** :

- le bucket S3 reste privé ;
- CloudFront est le seul service autorisé à accéder aux fichiers ;
- les utilisateurs accèdent à la documentation de manière plus sécurisée.

Question 7 : Comment CloudWatch aide-t-il à superviser une application serverless ?

Amazon CloudWatch collecte les métriques et les journaux (logs) des fonctions Lambda.

Il permet de :

- surveiller les performances ;
- détecter les erreurs ;
- analyser les exécutions ;
- configurer des alertes.

Si une fonction Lambda génère une **exception non gérée**, l'erreur est enregistrée dans les logs CloudWatch, ce qui aide les développeurs à identifier et corriger le problème.

**Question 8 : Pourquoi AWS Lambda atteint-elle ses limites avec un fichier de 50 Go ?
Quel service utiliser ?**

AWS Lambda est conçue pour des traitements courts et de taille limitée.

Un fichier de **50 Go** peut poser problème à cause :

- des limites de mémoire ;
- du temps maximal d'exécution ;
- des ressources de calcul disponibles.

Pour traiter de très gros volumes de données, il est préférable d'utiliser des services Big Data comme :

- AWS Glue pour le traitement et la transformation des données ;
- ou Amazon EMR pour exécuter des frameworks comme Hadoop ou Spark.

Ces services sont spécialement conçus pour gérer des fichiers volumineux et des traitements Big Data.

Table des matières

Introduction	3
Présentation de l'architecture	3
Services AWS utilisés	4
Infrastructure as Code (CloudFormation)	5
Déploiement de l'Infrastructure	7
Captures d'Écran	13
Conclusion	16
Table des matières	20